

IS-698 Homework 2 – UB01976

Task:

Building a prompt-based agent that, when given a user query and the list of 47 tool descriptions, performs the following:

1. **Tool Usage Awareness:** decide whether any tool is needed to answer the query.
2. **Tool Selection:** if a tool is needed, identify the correct tool from the 48 available options.

Evaluate your agent using 3 different LLMs of your choice.

Solution:

I have used Python in the google collab to evaluate the agent by using 3 different LLMs for this task (**llama3.2, mistral, and gemma2**). The Dataset consists of **20614 user queries** that must be processed and decided if there is a need for any tool for the user's query or not. In case the tool is required, then which tool among the **47 is required for that user query**.

LLM's Used – 3 LLM's are selected for this evaluation, **Llama 3.2 3b** which is a lightweight model which is efficient while maintaining very good reasoning. **Mistral 7b** is one of the strongest open-source models available, which is used to produce structured output. **Gemma2 2b** is the smallest model in our evaluation with architecture alternating between the local and global attention layers.

Prompt Design: I have designed the prompt in such a way that it can be run for all three models. Instead of asking the model to return in free text, I have restricted it to strictly return the response in JSON format so that it can be parsed. The prompt is designed in such a way, that the LLM model is asked to assume the role of assistant and pick the tool most suited for the query and return it. The prompt format is as shown below;

Prompt Template -

You are a resourceful intelligent assistant that decides whether a user's query requires an external tool or not.

You have been provided access to the following 47 tools:

1. FinanceTool: Stay informed with the latest financial updates...
2. ExchangeTool: Seamlessly convert currencies...
- ... [all 47 tools] ...

User Query: "<query>"

Respond ONLY with valid JSON:

```
{"tool_needed": "yes" or "no", "tool_name": "<ExactToolName> or null"}
```

Design Approach: I have proceeded with zero shot prompting, as no prior examples were given. So, all three models must predict if the tools are required for the queries or not based on the description given only. The model must also return the exact names of the tool, to prevent any sort of hallucination in names. **Temperature is set to 0**, to maintain consistency and have reproducible results across all the runs over different models.

Sampled Dataset Summary– The Meta tool benchmark dataset consists of 20614 user queries each labeled to a tool. The tool dataset consists of 47 tools with their respective descriptions. Every query in the dataset has been associated with one of the tools from the 47 available, and there is no query without tool requirement. This means every query has one or the other tool. The Sampling procedure is carried out in the following way;

Random_Seed = 42

For each of the 47 tools randomly sampled 5 queries (sample size $n=5$, Random_Seed = 42)

The reason for selecting 5 queries for each tool is that 2 is the minimum and at least required as per problem statement, but even if one of the two samples has been wrongly predicted then there would be a lot of change in the result and it would not be stable. Hence, with a **size of 5** the results can be much **more stable, and robust evaluation is possible**.

So, **Total Sampled** = 47 Tools X 5 Queries for each = 235

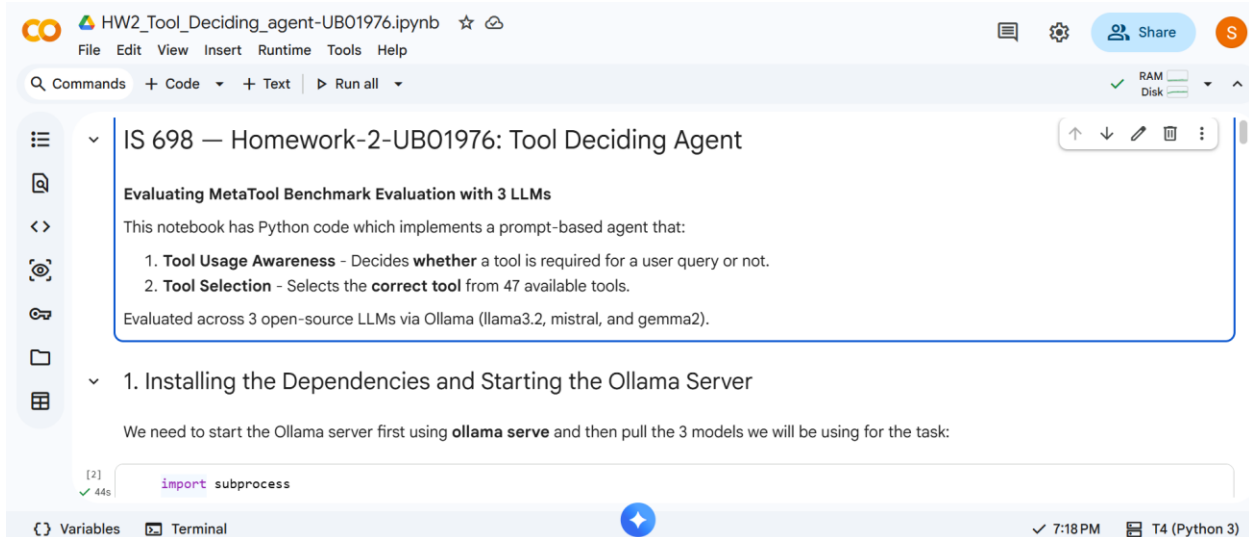
Hence, the original Dataset is of size 20,614 queries, but the **Sampled Dataset has a size of 235 queries**.

Dataset Limitation – Out of the 20,614 queries in the dataset, there is no specific query which is not associated with any of the tools. That means there are no false positives to

feed the LLM models (i.e. no queries with no tool needed or examples to let model get false positives). Hence, this limitation might change the evaluation metrics and give Precision as 1 which will be elaborately explained in the evaluation metrics.

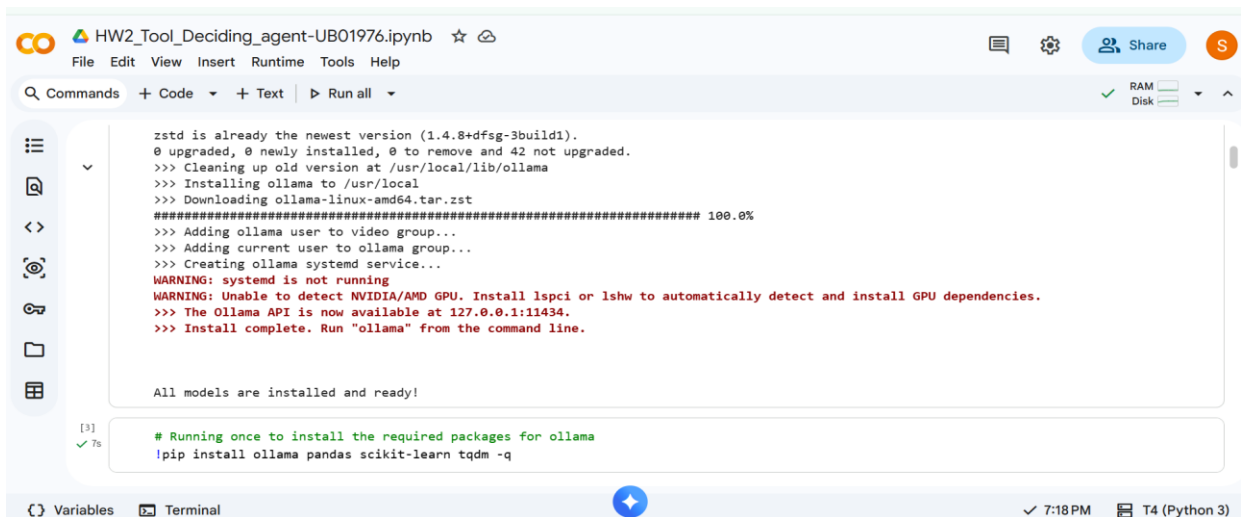
Code: The code is written in the Python and Executed in the Google Collab with T4 GPU's runtime Hardware (For Faster and better execution) and below are the screenshots of the code cells being executed;

First – Install all the 3 models using Ollama and Start the server



The screenshot shows a Google Colab notebook titled "HW2_Tool_Deciding_agent-UB01976.ipynb". The notebook has a menu bar with File, Edit, View, Insert, Runtime, Tools, and Help. Below the menu bar is a toolbar with icons for Commands, Code, Text, and Run all. The notebook content is divided into sections. The first section is titled "IS 698 — Homework-2-UB01976: Tool Deciding Agent". It contains a sub-section "Evaluating MetaTool Benchmark Evaluation with 3 LLMs" which states: "This notebook has Python code which implements a prompt-based agent that: 1. Tool Usage Awareness - Decides whether a tool is required for a user query or not. 2. Tool Selection - Selects the correct tool from 47 available tools. Evaluated across 3 open-source LLMs via Ollama (llama3.2, mistral, and gemma2)." The second section is titled "1. Installing the Dependencies and Starting the Ollama Server". It contains the text: "We need to start the Ollama server first using ollama serve and then pull the 3 models we will be using for the task:". Below this text is a code cell with the command `import subprocess`. The status bar at the bottom shows "Variables", "Terminal", "7:18 PM", and "T4 (Python 3)".

Install the necessary python libraries as well required for Ollama



The screenshot shows the same Google Colab notebook as before, but now the code cell is executing the installation of Ollama and its dependencies. The output of the code cell is as follows:

```
zstd is already the newest version (1.4.8+dfsg-3build1).
0 upgraded, 0 newly installed, 0 to remove and 42 not upgraded.
>>> Cleaning up old version at /usr/local/lib/ollama
>>> Installing ollama to /usr/local
>>> Downloading ollama-linux-amd64.tar.zst
##### 100.0%
>>> Adding ollama user to video group...
>>> Adding current user to ollama group...
>>> Creating ollama systemd service...
WARNING: systemd is not running
WARNING: Unable to detect NVIDIA/AMD GPU. Install lspci or lshw to automatically detect and install GPU dependencies.
>>> The Ollama API is now available at 127.0.0.1:11434.
>>> Install complete. Run "ollama" from the command line.

All models are installed and ready!
```

 Below this output is another code cell with the command `!pip install ollama pandas scikit-learn tqdm -q`. The status bar at the bottom shows "Variables", "Terminal", "7:18 PM", and "T4 (Python 3)".

Once it is done, we can go ahead with the Random Seed selection and load the configurations;

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text Run all

RAM Disk

2. Random Seed Selection for Reproducibility

```
[4] ✓ 5s
import json
import random
import pandas as pd
import numpy as np
from tqdm import tqdm
import ollama
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Reproducibility
RANDOM_SEED = 42
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

# Three open-source LLMs via Ollama
MODELS = [
    "llama3.2:3b", # Meta LLaMA 3.2 3B
```

Variables Terminal 7:20 PM T4 (Python 3)

Then, the respective Data file (all_clean_data.csv) and Tools File (big_tool_des.json) is loaded (Files need to be loaded every time the runtime gets disconnected)

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text Run all

RAM Disk

```
Models : ['llama3.2:3b', 'mistral:7b', 'gemma2:2b']
```

3. Loading Data related to tools (From big_tool_des.json) and Actual data File (all_clean_data.csv)

```
[6] ✓ 0s
# Tool data (47 tools) (Load the tools file in the Files section, and as this in Collab file will not be saved after runtime execution and w
with open("big_tool_des.json", "r") as f:
    TOOL_DESCRIPTIONS = json.load(f)

TOOL_NAMES = list(TOOL_DESCRIPTIONS.keys()) # 48 tool names

print(f"Loaded {len(TOOL_NAMES)} tool descriptions")
print("Tools:", TOOL_NAMES[:5], "...")

... Loaded 47 tool descriptions
Tools: ['FinanceTool', 'ExchangeTool', 'NewsTool', 'PolishTool', 'CharityTool'] ...
```

Variables Terminal 7:23 PM T4 (Python 3)

The screenshot shows a Jupyter Notebook titled "HW2_Tool_Deciding_agent-UB01976.ipynb". The code cell [1] contains the following Python code:

```
# Loading full dataset (Load the full data file in the Files section, and as this is in Collab file will not be saved after runtime execution)
full_df = pd.read_csv("all_clean_data.csv")
print(f"Full dataset shape: {full_df.shape}")
full_df.head(3)
```

The output of the code is a table with 3 rows and 2 columns: "Query" and "Tool".

	Query	Tool
0	Can I find academic research papers on this to...	ResearchHelper
1	Can I find any peer-reviewed papers?	ResearchHelper
2	Can I generate bibtex bibliographies?	ResearchHelper

Below the table, the text "4. Sampling the Dataset" is visible.

After the Datasets are loaded, the data file is sampled. I have selected the sample **size of 5**, because 2 is the minimum size of the sample and if even one of the samples goes wrong, the error percentage increases by 50% due to less size. So, having a size of **5 is better as it gives more reliable results and is statistically more accurate compared to a size of 2.**

The screenshot shows a Jupyter Notebook titled "HW2_Tool_Deciding_agent-UB01976.ipynb". The code cell [9] contains the following Python code:

```
SAMPLES_PER_TOOL = 5 # at least 2 required; we are using 5 for better and richer evaluation
sampled_rows = []
for tool in TOOL_NAMES:
    tool_rows = full_df[full_df["Tool"] == tool]
    n = min(SAMPLES_PER_TOOL, len(tool_rows))
    sampled_rows.append(tool_rows.sample(n=n, random_state=RANDOM_SEED))

sampled_df = pd.concat(sampled_rows).reset_index(drop=True)

print(f"Sampled dataset shape : {sampled_df.shape}")
print(f"Queries per tool      : {sampled_df['Tool'].value_counts().describe()}")

# Save the sampled dataset as a deliverable
```

After the dataset is sampled (Sampled into a dataset of size: 235 queries), then the prompt has been designed using a proper prompt template. The prompt design and Agent inference screenshots are as follows;

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

5. Prompt Design

I am using here a **single prompt** that asks the LLM to:

1. Decide if a tool is needed or not (yes or no)
2. If yes, then in the output give me the exact tool name from the list provided.

```
[11] ✓ 0s def build_tool_list_text(tool_descriptions: dict) -> str:
    """Format all 47 tools into a numbered list for the prompt."""
    lines = []
    for i, (name, desc) in enumerate(tool_descriptions.items(), 1):
        lines.append(f"{i}. {name}: {desc}")
    return "\n".join(lines)

TOOL_LIST_TEXT = build_tool_list_text(TOOL_DESCRIPTIONS)
```

Variables Terminal

7:29 PM T4 (Python 3)

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

```
[11] ✓ 0s # Preview the prompt for one query
sample_query = sampled_df.iloc[0]["Query"]
print(f"Sample query: {sample_query}")
print("\n" + "="*60)
print(build_prompt(sample_query)[:800], "...")
```

... Sample query: I want to know the comprehensive and up-to-date financial analysis, including factors such as revenue, expenses, profit margins

=====

You are a resourceful intelligent assistant that decides whether a user query requires calling an external tool or not.

You have been provided access to the following 47 tools:

1. FinanceTool: Stay informed with the latest financial updates, real-time insights, and analysis on a wide range of options, stocks, cryptoc
2. ExchangeTool: Seamlessly convert currencies with our integrated currency conversion tool.
3. NewsTool: Stay connected to global events with our up-to-date news around the world.
4. PolishTool: Elevate your content with our AI-powered tool, which utilizes advanced rewriting techniques to create more human-like expressi
5. CharityTool: Empower your charitable endeavors by accessing a comprehensive platform featuring no ...

Variables Terminal

7:29 PM T4 (Python 3)

The image displays two screenshots of a Jupyter Notebook interface, likely from a web browser. The top screenshot shows a notebook titled "HW2_Tool_Deciding_agent-UB01976.ipynb". The code in the cell is a function `parse_response` that takes `raw_text` and `tool_names` as input and returns a dictionary. The function's docstring describes its purpose: "Post-process the LLM output into structured fields." and lists a strategy: "1. Try to parse as clean JSON". The bottom screenshot shows a notebook titled "HW2_Tool_Deciding_agent-UB01976.ipynb" with a cell containing a function `run_evaluation`. This function takes a `model` and a `df` (pandas DataFrame) as input and returns a `pd.DataFrame`. The function's docstring describes its purpose: "Run the agent on all queries in df and return a results DataFrame." and lists a strategy: "For this benchmark all queries in df ARE tool-requiring (label = 1), because every row maps to a specific tool." The function prints the model name and the number of queries, and then iterates over the DataFrame rows to evaluate the model's performance.

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

6. Agent: LLM Inference + Post-processing

To make sure we have proper JSON for processing and do validation against tools available.

```
[14]
import re

def parse_response(raw_text: str, tool_names: list) -> dict:
    """
    Post-process the LLM output into structured fields.

    Strategy:
    1. Try to parse as clean JSON
```

Variables Terminal 7:32 PM T4 (Python 3)

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

7. Run Evaluation

```
[23]
def run_evaluation(model: str, df: pd.DataFrame) -> pd.DataFrame:
    """
    Run the agent on all queries in df and return a results DataFrame.

    For this benchmark all queries in df ARE tool-requiring (label = 1),
    because every row maps to a specific tool.
    """
    print(f"\n{' '*60}")
    print(f"Evaluating model: {model}")
    print(f"{' '*60}")

    results = []

    for idx, row in tqdm(df.iterrows(), total=len(df), desc=model):
        query = row["Query"]
```

Variables Terminal 7:37 PM T4 (Python 3)

Once the agent's functions are ready, it is run across all three models and the results from each model are saved in respective .csv files for analysis.

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[10]
✓ 11m
print("\nAll models evaluated!")

=====
Evaluating model: llama3.2:3b
=====
llama3.2:3b: 100%|██████████| 235/235 [03:32<00:00, 1.10it/s]
Saved → results_llama3.2_3b.csv

=====
Evaluating model: mistral:7b
=====
mistral:7b: 100%|██████████| 235/235 [04:19<00:00, 1.11s/it]
Saved → results_mistral_7b.csv

=====
Evaluating model: gemma2:2b
=====
gemma2:2b: 100%|██████████| 235/235 [03:42<00:00, 1.06it/s] Saved → results_gemma2_2b.csv

All models evaluated!
```

{ } Variables Terminal 11:52 PM T4 (Python 3)

Once all the models are evaluated, the evaluation metrics such as Accuracy, F1 score, Recall, CSR, and Precision are calculated for each model.

HW2_Tool_Deciding_agent-UB01976.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
[ ]
print(metrics_df['Accuracy'].idxmax())
print()
print(f"Best model by CSR : {metrics_df['CSR'].idxmax()} (CSR={metrics_df['CSR'].max():.4f})")
print(f"Best model by F1 : {metrics_df['F1 Score'].idxmax()} (F1={metrics_df['F1 Score'].max():.4f})")

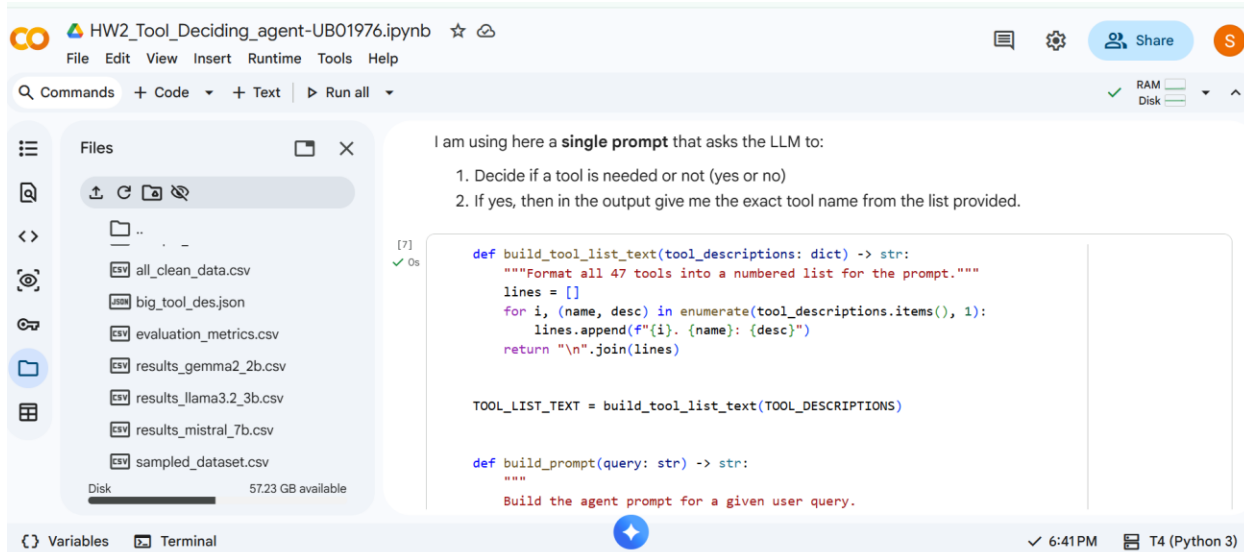
=====
FINAL RESULTS SUMMARY
=====
Random Seed : 42
Samples per tool : 5
Total evaluation rows : 235
Number of tools : 47

=====
Model Accuracy Precision Recall F1 Score CSR
llama3.2:3b 0.9447 1.0 0.9447 0.9716 0.6936
mistral:7b 0.9915 1.0 0.9915 0.9957 0.7362
gemma2:2b 0.9149 1.0 0.9149 0.9556 0.5872

Best model by CSR : mistral:7b (CSR=0.7362)
Best model by F1 : mistral:7b (F1=0.9957)
```

{ } Variables Terminal 11:52 PM

Result Analysis – Once all the models are run, the results of each model are saved in the respective .csv files which can be downloaded from the files.



Evaluation Metrics - Here I have considered metrics like F1 score, Accuracy, Recall, Precision, and CSR (Correct Selection Rate) for the evaluation of the agent. The Confusion matrix values for the three models are as follows;

	Gemma2 2b	Llama 3.2 3b	Mistral 7b
TP (True Positives)	215	222	233
FP (False Positives)	0	0	0
FN (False Negatives)	20	13	2
TN (True Negatives)	0	0	0

Analyzing each of those individually,

Accuracy: **Mistral 7b** has better accuracy compared to the remaining two models. Mistral 7b has an accuracy of 0.9915, and Llama 3b has 0.9447 and Gemma2 2b 0.9419 respectively. The primary reason why Gemma2 2b has lower accuracy is because it has 20 False Negatives (Model said No, but dataset has Tool) whereas Llama and Mistral have lower FN, as a result their accuracy is greater.

Recall: Coming to the Recall (i.e. actual tool use cases correctly identified) Mistral 7b again gets near to perfect recall score of 0.9915 again as it has got a smaller number of FN compared to Llama and Gemma2.

Precision: When it comes to precision, it is **1** for all 3 models. As mentioned earlier, this is because the dataset provided consists of only positive use case scenarios or queries where every query is associated or has a tool. There are no scenarios related to False Positives (FP) (i.e. actually False but predicted as positive).

F1 Score: The F1 score is basically dependent on the precision and recall, as it is their harmonic mean. As precision is 1, so F1 score basically becomes the scaled-up version of recall. **Mistral** obtained a F1 score of **0.9957**, which is higher compared to the remaining two models (i.e. Llama3.2 3b – 0.9716 and Gemma2 2b – 0.9556)

CSR (Correct Selection Rate): Now this metric tells whether the tool is correctly selecting the respective tools for the queries or not. It is basically a number of exact tools matches (i.e. whether the model selected the correct tool or not divided by total number of queries). Mistral 7b has CSR of **0.7362**, Llama3.2 3b is very nearer to it with CSR of **0.6936** and Gemma2 2b has **0.5872**. So, Mistral 7b is better among the three able to select the correct tool.

Below is the Evaluation Metrics Table;

	Gemma2 2b	Llama 3.2 3b	Mistral 7b
Accuracy	0.9149	0.9447	0.9915
Precision	1	1	1
F1 Score	0.9556	0.9716	0.9957
Recall	0.9149	0.9447	0.9915
CSR	0.5872	0.6936	0.7362

The GitHub Link to the Collab Notebook file, Evaluation metric, and Sampled Dataset is as follows;

https://github.com/UB01976/Software_Engineering_698/blob/main/HW2_Tool_Deciding_agent_UB01976.ipynb

Conclusion – Overall, built an agent that is using the zero-prompt technique to determine tool usage and awareness decision across three different LLM models – Llama3.2 3b, Mistral 7b, and Gemma2 2b using Ollama on the Metatool benchmark. The dataset of 20,614 is divided into samples of 235 (i.e. 5 queries for each tool of the 47 tools) to evaluate across different metrics. Out of all the 3 models, Mistral 7b can be considered as the strongest model based on Accuracy and Recall. Though Llama came very close to Mistral in terms of CSR, still Mistral has slightly greater CSR compared to the remaining two. When analyzed clearly, tools with better and unique descriptions achieved CSR of 1 across all three models such as MemeTool, CharityTool, and LawTool. Whereas some tools which had description overlapping with others caused confusion to the models such

as ResearchHelper, TripAdvice, and DataRetrievalTool. Hence, in conclusion, the tool's description plays a critical role in deciding whether a tool can be used for that query or not, especially in zero shot prompting.